

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОННОЙ ТЕХНИКИ

Институт системной и программной инженерии
и информационных технологий (Институт СПИНТех)

Лабораторная работа № 3

Создание однонаправленной нейронной сети

Выполнил:

Трусов М.П. гр. ПИН-42

Проверил преподаватель:

проф., д.ф.-м. н. Рычагов М.Н.

Москва, 2026

1. Подготовка данных

Для обучения и тестирования сети были сгенерированы данные.

Параметры генерации:

Количество точек функции: $N = 21$ ($x = 0, 0.05, \dots, 1.0$)

Диапазон параметров: $C, A, S \in [0.1, 1.0]$

Обучающая выборка: 500 примеров

Тестовая выборка: 200 примеров

Программный код:

```
def generate_data(m=100, n=21, noise=0.0):
    x = np.linspace(0, 1, n)
    p = np.zeros((m, n))
    t = np.zeros((m, 3))
    for i in range(m):
        c = 0.9 * np.random.rand() + 0.1
        a = 0.9 * np.random.rand() + 0.1
        s = 0.9 * np.random.rand() + 0.1
        y = c * np.exp(-((x - a) ** 2) / s)
        if noise > 0:
            y += noise * np.random.randn(n)
        p[i, :] = y
        t[i, :] = [c, a, s]
    return p, t, x
```

2. Создание сети

Была создана нейронная сеть прямого распространения со следующей архитектурой:

Входной слой: 21 нейрон (по числу точек функции)

Первый скрытый слой: 15 нейронов с функцией активации Sigmoid

Второй скрытый слой: 10 нейронов с функцией активации Tanh

Третий скрытый слой: 5 нейронов с функцией активации Tanh

Выходной слой: 3 нейрона с линейной функцией активации (для предсказания C, A, S)

Программный код:

```
def create_network(input_dim=21, hidden_units=(15, 10, 5)):
    model = Sequential()
    model.add(Input(shape=(input_dim,)))
    model.add(Dense(hidden_units[0], activation='sigmoid'))
    for units in hidden_units[1:]:
        model.add(Dense(units, activation='tanh'))
    model.add(Dense(3, activation='linear'))
    return model
```

3. Обучение сети

Параметры обучения:

Функция потерь: MSE

Оптимизатор: Adam с коэффициентом обучения 0.01

Размер батча: 32

Количество эпох обучения: 300

Валидационная выборка: 20 процентов от обучающей

Программный код:

```
def train_network(model, p_train, t_train, epochs=500, verbose=1):
    model.compile(optimizer=Adam(learning_rate=0.01), loss='mse',
metrics=['mae'])

    history = model.fit(p_train, t_train, epochs=epochs, batch_size=32,
validation_split=0.2,
verbose=verbose, shuffle=True)

    return history
```

4. Тестирование сети

Для оценки качества использовались коэффициент детерминации R^2 , коэффициент корреляции Пирсона, средняя абсолютная ошибка и среднеквадратичная ошибка.

Программный код:

```
def test_network(model, p_test, t_test):
    predictions = model.predict(p_test)
    results = {}
    param_names = ['C', 'A', 'S']

    for i, name in enumerate(param_names):
        true_vals = t_test[:, i]
        pred_vals = predictions[:, i]

        r2 = r2_score(true_vals, pred_vals)
        corr = np.corrcoef(true_vals, pred_vals)[0, 1]
        mae = np.mean(np.abs(true_vals - pred_vals))
        rmse = np.sqrt(np.mean((true_vals - pred_vals) ** 2))

        results[name] = {'r2': r2, 'correlation': corr, 'mae': mae,
                        'rmse': rmse,
                        'true': true_vals, 'pred': pred_vals}
    return results, predictions
```

5. Применение модели

Программный код:

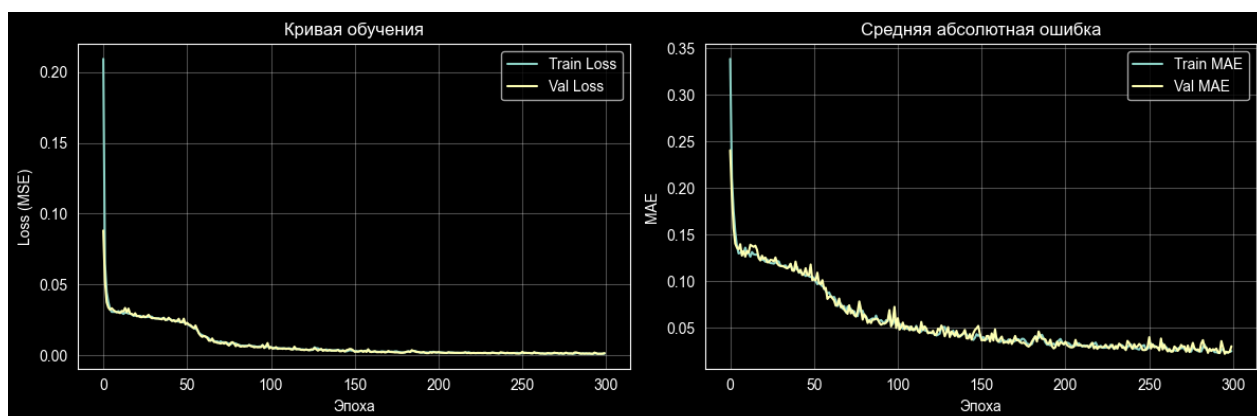
```
def predict_parameters(model, y_values):
    y_values = np.array(y_values).reshape(1, -1)
    prediction = model.predict(y_values)
    return {
        'C': prediction[0, 0],
        'A': prediction[0, 1],
        'S': prediction[0, 2]
    }
```

Вывод программы:

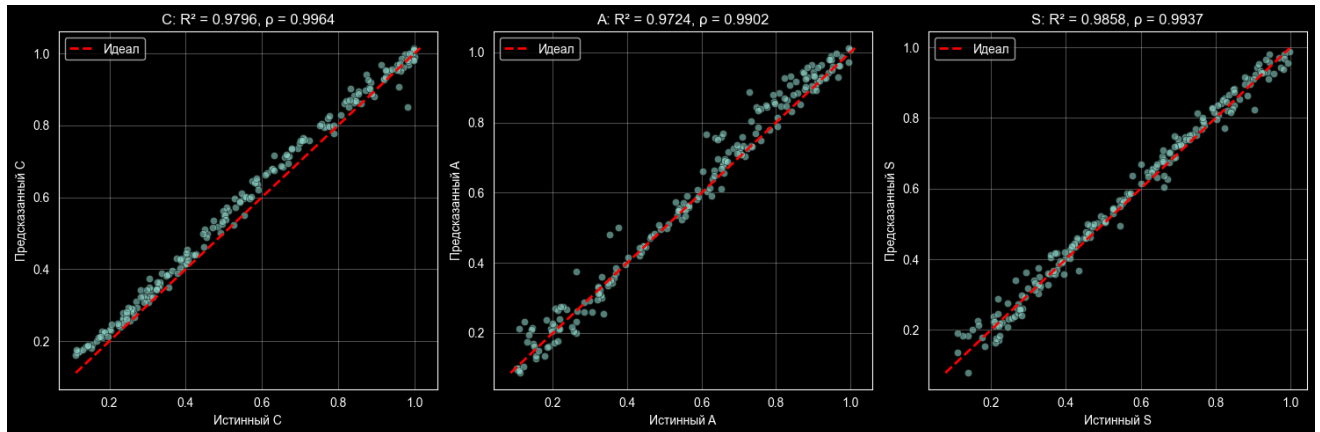
Истинные: C=0.6543, A=0.9947, S=0.8019

Предсказанные: C=0.6837, A=0.9717, S=0.7968

6. Кривая и результаты обучения

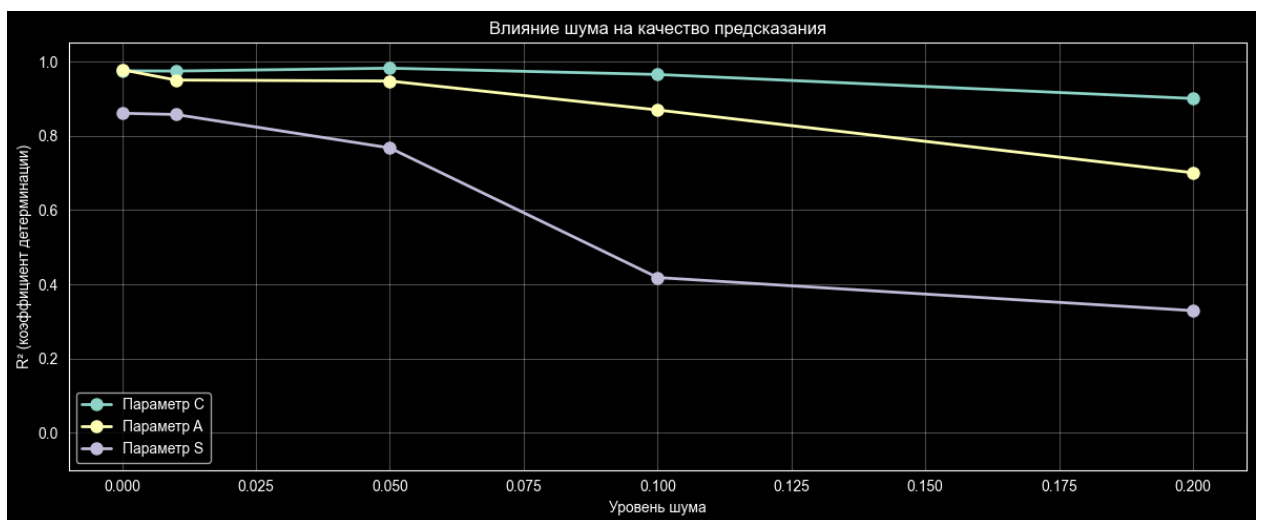


Параметр	R^2	Корреляция	MAE	RMSE
C	0.9796	0.9964	0.0340	0.0389
A	0.9724	0.9902	0.0317	0.0447
S	0.9858	0.9937	0.0235	0.0292



7. Исследование влияния шума

С увеличением уровня шума качество предсказания всех параметров снижается. Наиболее чувствительным к шуму оказался параметр S.



8. Эксперимент со случайными данными

Корреляция практически отсутствует.

Параметр	R^2	Корреляция	MAE	RMSE
C	-0.0718	0.1182	0.2482	0.2965
A	-0.0901	-0.0048	0.2637	0.3109
S	-0.3529	-0.0150	0.2836	0.3317

